

머신러닝

DAY1

학습 파이프라인과 데이터 준비

모델을 학습시키는 것보다 더 중요한 것은, 같은 절차를 누수 없이 반복하는 것이다.

개념 · 비교 · 결과해석 · 실전판단 · 복습

수업일 2026-08-11 · 원본 노트북 기반 재구성

1. 한눈에 보는 DAY

Iris와 Titanic 실습을 통해 scikit-learn의 공통 인터페이스, 데이터 분리, 교차검증, 범주형 인코딩을 연결한다. DAY1의 핵심은 특정 모델이 아니라 재사용 가능한 학습 순서다.

오늘의 핵심 질문

모델을 학습시키는 것보다 더 중요한 것은, 같은 절차를 누수 없이 반복하는 것이다.

문제 해결 흐름

1 문제와 목표변수 정의	2 학습/평가 데이터 분리	3 전처리 학습	4 모델 학습	5 검증 지표 해석
---------------------	----------------------	-------------	------------	---------------

핵심 키워드

fit / predict	train-test split	K-Fold	범주형 인코딩	Pipeline
---------------	------------------	--------	---------	----------

2. 핵심 개념

Estimator API

scikit-learn 모델은 대부분 fit으로 규칙을 학습하고 predict로 새 데이터의 결과를 만든다. 모델이 바뀌어도 호출 구조가 같기 때문에 실험을 비교하기 쉽다.

데이터 분리

평가 데이터는 마지막 시험지다. 전처리 통계까지 학습 데이터에서만 계산해야 하며 random_state는 재현 가능한 비교를 만든다. 분류에서는 클래스 비율을 유지하는 stratify가 유용하다.

교차검증

한 번의 분할 결과는 우연에 민감하다. K-Fold는 여러 분할의 평균과 변동을 함께 보여준다. 클래스가 불균형하면 StratifiedKFold를 우선 고려한다.

범주형 표현

LabelEncoder의 숫자는 크기 관계로 오해될 수 있다. 명목형 변수는 One-Hot, 순서가 실제 의미를 가질 때만 Ordinal 인코딩을 사용한다.

Pipeline

전처리와 모델을 한 객체로 묶으면 교차검증의 각 fold 안에서 전처리가 다시 학습된다. 이는 데이터 누수를 줄이는 가장 실용적인 습관이다.

3. 기법 비교와 선택

방법	표현	적합한 경우	주의점
Label encoding	범주를 정수 하나로 변환	목표 라벨 또는 순서가 명확한 범주	입력 특성에 쓰면 가짜 순서가 생길 수 있음
One-Hot	범주마다 0/1 열 생성	성별, 향구, 상품종류 같은 명목형	범주 수가 많으면 열이 크게 증가
Ordinal	지정한 순서대로 정수화	등급, 크기처럼 순서가 실제 의미를 가짐	순서를 데이터 의미에 맞게 직접 정의

노트북 실험에서 읽어야 할 것

노트북	무엇을 했나	어떻게 해석하나
01_sklearn	Iris를 분리하고 결정트리를 학습해 정확도를 계산	데이터 준비 - fit - predict - metric의 기본 뼈대를 확인
02_kfold	폴드마다 다른 학습/검증 인덱스와 정확도를 계산	평균만 보지 말고 fold 간 편차도 일반화 불확실성으로 해석
03_data	LabelEncoder, OneHotEncoder, pandas 변환 비교	숫자로 바꾸는 방식이 모델의 해석에 어떤 가정을 넣는지 구분
04_titanic	결측치와 범주형 변수를 처리한 뒤 세 분류기 비교	같은 전처리와 분할에서 모델을 비교해야 차이가 의미 있음

4. 실전 적용 기준

판단 체크리스트

데이터가 작거나 분할에 민감하면 단일 hold-out보다 교차검증을 사용한다.

문자열 범주는 학습 데이터에서 encoder를 fit하고 평가 데이터에는 transform만 적용한다.

모델 비교에서는 전처리, 데이터 분할, 평가 지표를 고정한다.

범주형 변수에 실제 순서가 없으면 One-Hot을 기본 선택으로 본다.

최소 코드 패턴

```
preprocess = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("num", SimpleImputer(strategy="median"), num_cols),
])
model = Pipeline([("prep", preprocess),
                  ("clf", LogisticRegression(max_iter=1000))])
scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
```

→ 핵심은 코드 길이가 아니라 전처리가 fold 안에서 학습된다는 점이다.

→ 새 범주가 들어와도 handle_unknown 설정으로 평가 단계가 깨지지 않는다.

5. 실수 방지와 복습

자주 생기는 오류

- 전체 데이터로 인코더나 스케일러를 먼저 fit한 뒤 교차검증하기
- LabelEncoder를 모든 입력 범주에 습관적으로 사용하기
- 서로 다른 데이터 분할의 점수를 모델 차이로 해석하기
- Titanic의 ID, 이름처럼 목표와 무관하거나 누수를 만들 수 있는 열을 그대로 사용하기

스스로 설명해 보기

- ☐ K-Fold와 StratifiedKFold의 차이를 설명할 수 있는가?
- ☐ fit_transform과 transform을 학습/평가 데이터에 어떻게 적용해야 하는가?
- ☐ One-Hot 인코딩의 장점과 차원 증가 문제를 설명할 수 있는가?
- ☐ Pipeline이 데이터 누수를 줄이는 이유는 무엇인가?

DAY 핵심 요약

1	좋은 모델 비교의 출발점은 같은 데이터, 같은 전처리, 같은 지표다.
2	평가 데이터는 모델뿐 아니라 전처리에도 보여주지 않는다.
3	DAY1의 최종 산출물은 모델 하나가 아니라 재현 가능한 파이프라인이다.

원본 노트북의 전체 코드는 기존 코드 중심 PDF에서 확인하고, 이 자료는 개념 이해와 결과 해석을 위한 복습본으로 사용하세요.